Practical-6

Problem Definition

Formulate and test an automated storage lifecycle policy matrix defining three distinct access tiers:

- Hot – Immediate access
- Warm – Infrequent access
- Cold – Deep or archival access

Students will implement programmatic lifecycle rules that automatically transition data objects across these storage tiers based on retention targets, query-frequency boundaries, and storage costs.

The practical focuses on designing automated storage lifecycle policies that optimize storage cost while maintaining appropriate access performance. Students will work with simulated operational logs, access-time manifests, and server audit records representing historical data across multiple time periods.

The lifecycle policies will be implemented and tested using MinIO Client (MC), MinIO lifecycle engine, JSON lifecycle policy schemas, and Python automation scripts.

Dataset

The dataset/test data consists of simulated historical storage and access records.

The records include:

- Historical system operational logs
- Data object access-time manifests
- Server audit records
- Object retention information
- Query/access frequency information
- Storage tier information
- Object creation and modification timestamps
- Historical access patterns

The test data is used to determine when objects should remain in Hot storage, transition to Warm storage, or move to Cold storage based on predefined lifecycle conditions.

Code:

1. cost_analysis_calculator.py

```
import json
import os
import matplotlib.pyplot as plt


# Tier Pricing Rates ($ per GB per month)
PRICING = {
    "Hot": 0.0230,    # High Throughput NVMe/SSD
    "Warm": 0.0125,   # Infrequent Access HDD
    "Cold": 0.00099,  # Deep Compression Archiving
    "EXPIRED (DELETED)": 0.0000
}


def analyze_cost_reductions(manifest_before="outputs/dataset_manifest.json", manifest_after="outputs/updated_dataset_manifest.json"):
    os.makedirs("outputs", exist_ok=True)

    if not os.path.exists(manifest_after):
        from lifecycle_automation import evaluate_and_enforce_lifecycle
        evaluate_and_enforce_lifecycle()

    with open(manifest_before, "r") as f:
        before_objects = json.load(f)

    with open(manifest_after, "r") as f:
        after_objects = json.load(f)
```

```
    # 1. Baseline Cost (All data retained in HOT tier)
    total_gb = sum(obj["size_gb"] for obj in before_objects)
    baseline_monthly_cost = total_gb * PRICING["Hot"]
    baseline_annual_cost = baseline_monthly_cost * 12

    # 2. Optimized Tiered Cost
    optimized_monthly_cost = 0.0
    tier_gb_breakdown = {"Hot": 0.0, "Warm": 0.0, "Cold": 0.0, "EXPIRED (DELETED)": 0.0}

    for obj in after_objects:
        tier = obj["current_tier"]
        size = obj["size_gb"]
        tier_gb_breakdown[tier] += size
        optimized_monthly_cost += size * PRICING.get(tier, 0.0)

    optimized_annual_cost = optimized_monthly_cost * 12

    monthly_savings = baseline_monthly_cost - optimized_monthly_cost
    annual_savings = baseline_annual_cost - optimized_annual_cost
    pct_reduction = (monthly_savings / baseline_monthly_cost) * 100

    print("=" * 80)
      print("              FINANCIAL COST REDUCTION & STORAGE OPTIMIZATION
REPORT          ")
    print("=" * 80)
    print(f" Total Dataset Capacity Evaluated: {total_gb:,.1f} GB ({total_gb/1024:.2f} TB)\n")
    print(" [1] BASELINE STORAGE COSTS (Unmanaged / All Hot Tier):")
    print(f"     - Monthly Baseline Cost:  ${baseline_monthly_cost:,.2f}")
```

```
    print(f"    - Annual Baseline Cost:   ${baseline_annual_cost:,.2f}\n")
    print(" [2] OPTIMIZED TIERED STORAGE COSTS (Automated ILM Matrix):")
        print(f"        - Hot Tier Storage ({tier_gb_breakdown['Hot']:,.1f} GB @ $0.023/GB):      ${tier_gb_breakdown['Hot']*PRICING['Hot']:,.2f}")
        print(f"        - Warm Tier Storage ({tier_gb_breakdown['Warm']:,.1f} GB @ $0.0125/GB):   ${tier_gb_breakdown['Warm']*PRICING['Warm']:,.2f}")
        print(f"        - Cold Tier Storage ({tier_gb_breakdown['Cold']:,.1f} GB @ $0.00099/GB):  ${tier_gb_breakdown['Cold']*PRICING['Cold']:,.2f}")
    print(f"    - Expired Data Purged ({tier_gb_breakdown['EXPIRED (DELETED)']:,.1f} GB @ $0.00):      $0.00")
    print(f"    - Total Monthly Optimized Cost: ${optimized_monthly_cost:,.2f}")
    print(f"    - Total Annual Optimized Cost:  ${optimized_annual_cost:,.2f}\n")
    print(" [3] SAVINGS SUMMARY:")
     print(f"      - Monthly Cost Reduction: ${monthly_savings:,.2f} ({pct_reduction:.2f}% savings)")
    print(f"    - Annual Cost Reduction:  ${annual_savings:,.2f} / year saved!\n")
    print("=" * 80)


    # Plot Cost Reduction Chart
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))


    # Bar chart: Baseline vs Optimized
    categories = ['Unmanaged Baseline (Hot Only)', 'Automated Tiered Lifecycle']
    costs = [baseline_monthly_cost, optimized_monthly_cost]
    colors = ['#e74c3c', '#2ecc71']


    bars = ax1.bar(categories, costs, color=colors, width=0.5)
    ax1.set_ylabel('Monthly Cost ($ USD)', fontsize=12, fontweight='bold')
        ax1.set_title('Monthly Operational Storage Cost Comparison', fontsize=13, fontweight='bold')
    ax1.grid(axis='y', linestyle='--', alpha=0.7)
```

```
    for bar in bars:
        yval = bar.get_height()
            ax1.text(bar.get_x() + bar.get_width()/2.0, yval + 1, f"${yval:,.2f}", ha='center', va='bottom', fontweight='bold', fontsize=11)

    # Pie Chart: Storage Capacity Distribution by Tier
    tier_names = [k for k, v in tier_gb_breakdown.items() if v > 0]
    tier_sizes = [v for k, v in tier_gb_breakdown.items() if v > 0]
    pie_colors = ['#ff7675', '#ffeaa7', '#74b9ff', '#b2bec3']

            ax2.pie(tier_sizes, labels=tier_names, autopct='%1.1f%%', startangle=140, colors=pie_colors, textprops={'fontweight': 'bold'})
     ax2.set_title(f'Post-Policy Capacity Distribution (Total: {total_gb:,.0f} GB)', fontsize=13, fontweight='bold')

    plt.tight_layout()
    chart_path = "outputs/cost_reduction_chart.png"
    plt.savefig(chart_path, dpi=300)
    plt.close()

    print(f"[+] Cost reduction breakdown graph generated and saved to: {chart_path}")

    # Generate Markdown Summary Report
     report_md = f"""# Practical 6: Automated Storage Lifecycle Policy - Financial Cost Reduction Breakdown

## Executive Cost Reduction Summary
- **Evaluated Data Capacity**: {total_gb:,.1f} GB ({total_gb/1024:.2f} TB)
- **Baseline Monthly Cost (All Hot)**: ${baseline_monthly_cost:,.2f}
```

```
- **Optimized Monthly Cost (Tiered ILM)**: ${optimized_monthly_cost:,.2f}

- **Monthly Net Savings**: **${monthly_savings:,.2f}** ({pct_reduction:.2f}% Cost Reduction)

- **Annualized Net Savings**: **${annual_savings:,.2f} / year**


## Capacity Breakdown by Storage Class

| Access Tier | Capacity (GB) | Percentage (%) | Unit Cost ($/GB/mo) | Monthly Cost ($) |

|---|---|---|---|---|

| **Hot (NVMe / SSD)** | {tier_gb_breakdown['Hot']:,.1f} GB | {(tier_gb_breakdown['Hot']/total_gb)*100:.1f}% | $0.0230 | ${tier_gb_breakdown['Hot']*PRICING['Hot']:,.2f} |

| **Warm (Infrequent Access)** | {tier_gb_breakdown['Warm']:,.1f} GB | {(tier_gb_breakdown['Warm']/total_gb)*100:.1f}% | $0.0125 | ${tier_gb_breakdown['Warm']*PRICING['Warm']:,.2f} |

| **Cold (Deep Archive)** | {tier_gb_breakdown['Cold']:,.1f} GB | {(tier_gb_breakdown['Cold']/total_gb)*100:.1f}% | $0.00099 | ${tier_gb_breakdown['Cold']*PRICING['Cold']:,.2f} |

| **Purged (Expired Temp Data)** | {tier_gb_breakdown['EXPIRED (DELETED)']:,.1f} GB | {(tier_gb_breakdown['EXPIRED (DELETED)']/total_gb)*100:.1f}% | $0.0000 | $0.00 |

| **TOTAL** | **{total_gb:,.1f} GB** | **100.0%** | -- | **${optimized_monthly_cost:,.2f}** |

"""

    with open("outputs/cost_reduction_report.md", "w") as f:

        f.write(report_md)


    return baseline_monthly_cost, optimized_monthly_cost, monthly_savings, pct_reduction


if __name__ == "__main__":

    analyze_cost_reductions()
```

2. dataset_generator.py

```
import json
import csv
import os
from datetime import datetime, timedelta, timezone

def generate_datasets(output_dir="outputs"):
    os.makedirs(output_dir, exist_ok=True)

    # Reference date representing current system timestamp
    current_time = datetime(2026, 10, 2, 10, 0, 0, tzinfo=timezone.utc)

    # Simulated storage objects across 4 quarters
    objects_data = [
        {
            "object_id": "obj_001",
            "object_name": "db_tables/live_orders_2026.parquet",
            "size_gb": 450.0,
            "created_at": (current_time - timedelta(days=12)).isoformat(),
            "last_accessed_at": (current_time - timedelta(hours=2)).isoformat(),
            "days_since_last_access": 0,
            "total_access_count_30d": 3450,
            "current_tier": "Hot",
            "content_type": "transactional_db"
        },
        {
            "object_id": "obj_002",
            "object_name": "db_tables/user_profiles_v2.parquet",
            "size_gb": 120.0,
            "created_at": (current_time - timedelta(days=25)).isoformat(),
```

```
        "last_accessed_at": (current_time - timedelta(days=1)).isoformat(),
        "days_since_last_access": 1,
        "total_access_count_30d": 1200,
        "current_tier": "Hot",
        "content_type": "transactional_db"
    },
    {
        "object_id": "obj_003",
        "object_name": "logs/app_server_q2_2026.log",
        "size_gb": 850.0,
        "created_at": (current_time - timedelta(days=40)).isoformat(),
        "last_accessed_at": (current_time - timedelta(days=35)).isoformat(),
        "days_since_last_access": 35,
        "total_access_count_30d": 4,
        "current_tier": "Hot",
        "content_type": "operational_log"
    },
    {
        "object_id": "obj_004",
        "object_name": "db_tables/historical_sales_q1_2026.csv",
        "size_gb": 620.0,
        "created_at": (current_time - timedelta(days=110)).isoformat(),
        "last_accessed_at": (current_time - timedelta(days=95)).isoformat(),
        "days_since_last_access": 95,
        "total_access_count_30d": 0,
        "current_tier": "Warm",
        "content_type": "historical_table"
    },
    {
```

```
        "object_id": "obj_005",
        "object_name": "db_tables/customer_feedback_2025.csv",
        "size_gb": 310.0,
        "created_at": (current_time - timedelta(days=140)).isoformat(),
        "last_accessed_at": (current_time - timedelta(days=120)).isoformat(),
        "days_since_last_access": 120,
        "total_access_count_30d": 0,
        "current_tier": "Hot",  # Misclassified hot object needing demotion!
        "content_type": "historical_table"
    },
    {
        "object_id": "obj_006",
        "object_name": "audit/compliance_records_2025_q4.tar.gz",
        "size_gb": 1250.0,
        "created_at": (current_time - timedelta(days=220)).isoformat(),
        "last_accessed_at": (current_time - timedelta(days=180)).isoformat(),
        "days_since_last_access": 180,
        "total_access_count_30d": 0,
        "current_tier": "Warm",
        "content_type": "audit_archive"
    },
    {
        "object_id": "obj_007",
        "object_name": "audit/security_scans_2025_q3.tar.gz",
        "size_gb": 980.0,
        "created_at": (current_time - timedelta(days=310)).isoformat(),
        "last_accessed_at": (current_time - timedelta(days=270)).isoformat(),
        "days_since_last_access": 270,
        "total_access_count_30d": 0,
```

```
        "current_tier": "Cold",
        "content_type": "audit_archive"
    },
    {
        "object_id": "obj_008",
        "object_name": "temp/staging_etl_dump_2025.tmp",
        "size_gb": 540.0,
        "created_at": (current_time - timedelta(days=400)).isoformat(),
        "last_accessed_at": (current_time - timedelta(days=390)).isoformat(),
        "days_since_last_access": 390,
        "total_access_count_30d": 0,
        "current_tier": "Cold",
        "content_type": "temporary_dump"
    }
]

# Save manifest JSON
manifest_path = os.path.join(output_dir, "dataset_manifest.json")
with open(manifest_path, "w") as f:
    json.dump(objects_data, f, indent=4)

# Save system access log CSV
csv_path = os.path.join(output_dir, "system_access_logs.csv")
with open(csv_path, "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=[
        "object_id", "object_name", "size_gb", "created_at",
        "last_accessed_at", "days_since_last_access", "total_access_count_30d",
        "current_tier", "content_type"
    ])
```

```
        writer.writeheader()
        writer.writerows(objects_data)

    print(f"[+] Dataset successfully generated!")
    print(f"    - Manifest file: {manifest_path}")
    print(f"    - Operational access log: {csv_path}")

if __name__ == "__main__":
    generate_datasets()
```

3. demote_inactive_tables.py

```
    import json
    import csv
    import os

    def scan_and_demote_inactive_tables(manifest_file="outputs/dataset_manifest.json"):
        print("=" * 80)
            print("          SUPPLEMENTARY AUTOMATION: INACTIVE TABLE TIER
    DEMOTION API ENGINE     ")
        print("=" * 80)

        if not os.path.exists(manifest_file):
            print(f"[-] Manifest file not found at {manifest_file}. Generating dataset first...")
            from dataset_generator import generate_datasets
            generate_datasets()

        with open(manifest_file, "r") as f:
            objects = json.load(f)
```

```
    flagged_tables = []
    api_call_audit_trail = []

    print(f"\n[1] Scanning data access logs for table entities with Inactive Days > 90...\n")

    for obj in objects:
        is_table = obj["content_type"] in ["historical_table", "transactional_db"] or obj["object_name"].startswith("db_tables/")
        days_inactive = obj["days_since_last_access"]

        if is_table and days_inactive > 90:
            flagged_tables.append(obj)

            # Simulate MinIO / AWS S3 API Call for Storage Class Demotion
            api_endpoint = "https://minio.internal.datacenter:9000"
            bucket_name = "enterprise-datalake"
            object_key = obj["object_name"]
            target_storage_class = "COLD_DEEP_ARCHIVE"

            # API payload structure
            api_call = {
                "timestamp": "2026-10-02T10:15:00Z",
                "api_action": "PutObjectStorageClass",
                "bucket": bucket_name,
                "key": object_key,
                "previous_storage_tier": obj["current_tier"],
                "new_storage_tier": target_storage_class,
                "status": "200 OK (SUCCESS)",
```

```
            "demotion_reason": f"Table inactive for {days_inactive} days (> 90 days retention threshold)"
        }
        api_call_audit_trail.append(api_call)

        print(f"  [FLAGGED] Table: {obj['object_name']}")
        print(f"          - Size: {obj['size_gb']} GB | Days Unread: {days_inactive}")
            print(f"          - Current Tier: {obj['current_tier']} ---> Demoting to: {target_storage_class}")
                print(f"          - Executing API Call: POST {api_endpoint}/{bucket_name}/{object_key}?x-minio-storage-class={target_storage_class}")
        print(f"          - Response: 200 OK | Storage Tier Successfully Demoted!\n")

    # Write audit log to output file
    audit_file = "outputs/inactive_tables_api_audit.json"
    with open(audit_file, "w") as f:
        json.dump(api_call_audit_trail, f, indent=4)

    print("=" * 80)
    print(f" SUMMARY REPORT:")
    print(f"  - Total Tables Scanned: {len(objects)}")
    print(f"  - Inactive Tables Flagged (>90 Days): {len(flagged_tables)}")
    print(f"  - API Demotion Commands Executed: {len(api_call_audit_trail)}")
    print(f"  - Audit Trail Saved: {audit_file}")
    print("=" * 80)

    return flagged_tables

if __name__ == "__main__":
```

```
    scan_and_demote_inactive_tables()
```

4. lifecycle_automation.py

```
import json
import csv
import os
from datetime import datetime

def evaluate_and_enforce_lifecycle(input_file="outputs/dataset_manifest.json", output_file="outputs/lifecycle_execution_log.txt"):
    os.makedirs("outputs", exist_ok=True)

    if not os.path.exists(input_file):
        print(f"[-] Error: {input_file} not found. Running dataset generator first...")
        from dataset_generator import generate_datasets
        generate_datasets()

    with open(input_file, "r") as f:
        objects = json.load(f)

    log_entries = []
    log_entries.append("================================================================================")
    log_entries.append("                AUTOMATED STORAGE LIFECYCLE POLICY ENFORCEMENT ENGINE            ")
    log_entries.append(f" Execution Timestamp: {datetime.now().strftime('%Y-%m-%d %H:%M:%S UTC')}")
    log_entries.append("================================================================================\n")
```

```
    transitioned_objects = []


    for obj in objects:
        original_tier = obj["current_tier"]
        days_inactive = obj["days_since_last_access"]
        size_gb = obj["size_gb"]
        name = obj["object_name"]


        target_tier = original_tier
        action = "NO_CHANGE"
        rationale = "Object access pattern within operational threshold."


        # Lifecycle Matrix Evaluation Logic:
        # 1. If days_since_last_access >= 365 and content_type == 'temporary_dump' -> EXPIRE / DELETE
        if days_inactive >= 365 and "temp/" in name:
            target_tier = "EXPIRED (DELETED)"
            action = "EXPIRE_OBJECT"
            rationale = "Exceeded 365-day retention policy limit for temporary/staging objects."
                # 2. If days_since_last_access >= 180 or (days_inactive >= 90 and obj['total_access_count_30d'] == 0) -> COLD
        elif days_inactive >= 180 or (days_inactive >= 90 and obj["total_access_count_30d"] == 0):
            if original_tier != "Cold":
                target_tier = "Cold"
                action = "TRANSITION_TO_COLD"
                 rationale = f"Inactive for {days_inactive} days (>90 day boundary with 0 queries). Demoted to Deep Compression Archiving."
        # 3. If days_since_last_access >= 30 and original_tier == "Hot" -> WARM
        elif days_inactive >= 30 and original_tier == "Hot":
```

```
            target_tier = "Warm"
            action = "TRANSITION_TO_WARM"
            rationale = f"Inactive for {days_inactive} days (>30 day threshold). Demoted from Hot to Warm Infrequent Access."

        obj_copy = dict(obj)
        obj_copy["previous_tier"] = original_tier
        obj_copy["current_tier"] = target_tier
        obj_copy["action_taken"] = action
        obj_copy["rationale"] = rationale
        transitioned_objects.append(obj_copy)

        status_flag = "[TRANSITION]" if action != "NO_CHANGE" else "[NO CHANGE]"
        entry = (
            f"{status_flag} Object: {name}\n"
            f"    - Size: {size_gb:.1f} GB | Inactive Days: {days_inactive} | 30d Query Count: {obj['total_access_count_30d']}\n"
            f"    - Tier Transition: {original_tier} ---> {target_tier}\n"
            f"    - Policy Rationale: {rationale}\n"
        )
        log_entries.append(entry)
        print(entry)

    # Save execution log
    with open(output_file, "w") as f:
        f.write("\n".join(log_entries))

    # Save updated manifest
    with open("outputs/updated_dataset_manifest.json", "w") as f:
        json.dump(transitioned_objects, f, indent=4)
```

```
    print(f"[+] Lifecycle policy enforcement complete. Detailed log saved to: {output_file}")
    return transitioned_objects

if __name__ == "__main__":
    evaluate_and_enforce_lifecycle()
```

5. mc_simulator.py

```
import sys
import json
import os

def mc_cli(args):
    if len(args) < 1:
        print("Usage: python mc_simulator.py [ilm|ls|stat|admin] [subcommands...]")
        return

    cmd = args[0].lower()

    if cmd == "ilm":
        subcmd = args[1] if len(args) > 1 else "ls"
        if subcmd in ["rule", "ls"]:
            print("mc: Lifecycle rules configured on alias 'myminio/enterprise-datalake':")
            print("-" * 85)
                print(f"{'RULE ID':<35} {'STATUS':<10} {'PREFIX':<15} {'TRANSITION / EXPIRATION'}")
            print("-" * 85)
            policy_file = "ilm_lifecycle_policy.json"
```

```
        if os.path.exists(policy_file):
            with open(policy_file, "r") as f:
                policy = json.load(f)
            for rule in policy.get("Rules", []):
                rule_id = rule.get("ID", "")
                status = rule.get("Status", "")
                prefix = rule.get("Filter", {}).get("Prefix", "")
                transitions = rule.get("Transitions", [])
                exp = rule.get("Expiration", {})
                t_str = ""
                if transitions:
                    t_str = ", ".join([f"{t['Days']}d -> {t['StorageClass']}" for t in transitions])
                elif exp:
                    t_str = f"Expire after {exp.get('Days')} days"
                print(f"{rule_id:<35} {status:<10} {prefix:<15} {t_str}")
        print("-" * 85)
    elif subcmd in ["import", "add"]:
        target_file = args[2] if len(args) > 2 else "ilm_lifecycle_policy.json"
         print(f"mc: Successfully imported lifecycle policy rule matrix from '{target_file}' to bucket 'myminio/enterprise-datalake'.")
        print("mc: Enforcing ILM rules on active object instances...")
    elif subcmd == "export":
        print("mc: Exporting active ILM policy rules JSON schema:")
        with open("ilm_lifecycle_policy.json", "r") as f:
            print(f.read())
elif cmd == "ls":
    print("mc: Listing objects in bucket 'myminio/enterprise-datalake':")
                        manifest_file = "outputs/updated_dataset_manifest.json" if os.path.exists("outputs/updated_dataset_manifest.json") else "outputs/dataset_manifest.json"
    if os.path.exists(manifest_file):
```

```
        with open(manifest_file, "r") as f:
            objects = json.load(f)
        print(f"{'DATE':<20} {'SIZE':<10} {'STORAGE TIER':<15} {'OBJECT NAME'}")
        print("-" * 75)
        for obj in objects:
            created = obj['created_at'][:19].replace('T', ' ')
            size = f"{obj['size_gb']} GiB"
            tier = obj['current_tier']
            name = obj['object_name']
            print(f"{created:<20} {size:<10} {tier:<15} {name}")

if __name__ == "__main__":
    mc_cli(sys.argv[1:])
```

6. run_practical_6.py

```
 import sys
import os
import time

def run_pipeline():
    print("=" * 85)
    print("        PRACTICAL 6: AUTOMATED STORAGE LIFECYCLE POLICY MATRIX EXECUTION      ")
    print("=" * 85)
    time.sleep(0.5)

    print("\n[STEP 1] Generating Historical System Operational Logs & Access Manifests...")
    from dataset_generator import generate_datasets
    generate_datasets()
```

```
    time.sleep(0.5)

    print("\n[STEP 2] Inspecting MinIO JSON Lifecycle Policy Schema (ILM CLI)...")
    from mc_simulator import mc_cli
    mc_cli(["ilm", "import", "ilm_lifecycle_policy.json"])
    print()
    mc_cli(["ilm", "ls"])
    time.sleep(0.5)

    print("\n[STEP 3] Running Background Lifecycle Automation Engine (Evaluating Retention & Access)...")
    from lifecycle_automation import evaluate_and_enforce_lifecycle
    evaluate_and_enforce_lifecycle()
    time.sleep(0.5)

    print("\n[STEP 4] Executing Supplementary Automation: Inactive Data Table API Demotion (>90 Days)...")
    from demote_inactive_tables import scan_and_demote_inactive_tables
    scan_and_demote_inactive_tables()
    time.sleep(0.5)

    print("\n[STEP 5] Calculating Cost Reductions & Generating Financial Charts...")
    from cost_analysis_calculator import analyze_cost_reductions
    analyze_cost_reductions()
    time.sleep(0.5)

    print("\n[STEP 6] Final MinIO Storage Object State & Tier Audit:")
    mc_cli(["ls"])

    print("\n" + "=" * 85)
```

```
    print(" [SUCCESS] PRACTICAL 6 COMPLETE! All outputs, logs, and charts saved to 'outputs/'.")
    print("=" * 85)


if __name__ == "__main__":
    run_pipeline()
```

Key Questions / Analysis / Interpretation

Que / Key Point 1: Detail the hardware and software properties that differentiate high-throughput Hot tier storage pools from highly compressed Cold tier archiving solutions.

Hot storage is designed for data requiring immediate and frequent access. It generally uses high-performance storage resources and provides lower access latency and higher throughput.

Cold storage is designed for historical or rarely accessed data. It can use highly compressed or archival-oriented storage mechanisms to reduce storage costs.

Therefore, the major differences include:

- Access frequency
- Access latency
- Storage performance
- Compression level
- Storage cost
- Retention duration
- Query requirements

Hot storage prioritizes performance, while Cold storage primarily prioritizes cost efficiency and long-term retention.

Que / Key Point 2: What are the typical operational and financial cost penalties incurred when an enterprise misclassifies historical active data into Cold states prematurely?

If frequently accessed historical data is moved to a Cold storage tier prematurely, applications may experience slower data retrieval and additional operational overhead.

Possible penalties include:

- Increased retrieval latency

- Reduced query performance
- Additional retrieval operations
- Increased operational complexity
- Higher data access/retrieval costs
- Delays in analytical workloads
- Poor user experience for applications requiring historical data

Therefore, lifecycle policies must consider both storage cost and access frequency before transitioning objects to Cold storage.

Que / Key Point 3: Explain how automated expiration policies safeguard database systems against storage performance degradation over prolonged windows.

Automated expiration policies automatically remove objects that have exceeded their required retention period.

This helps prevent unnecessary accumulation of old data and controls storage growth over time.

Automated expiration can:

- Remove obsolete objects
- Reduce unnecessary storage consumption
- Maintain manageable bucket sizes
- Reduce metadata growth
- Prevent excessive storage utilization
- Improve long-term storage management
- Enforce organizational retention requirements

Therefore, automated expiration policies help maintain efficient storage utilization over prolonged periods.

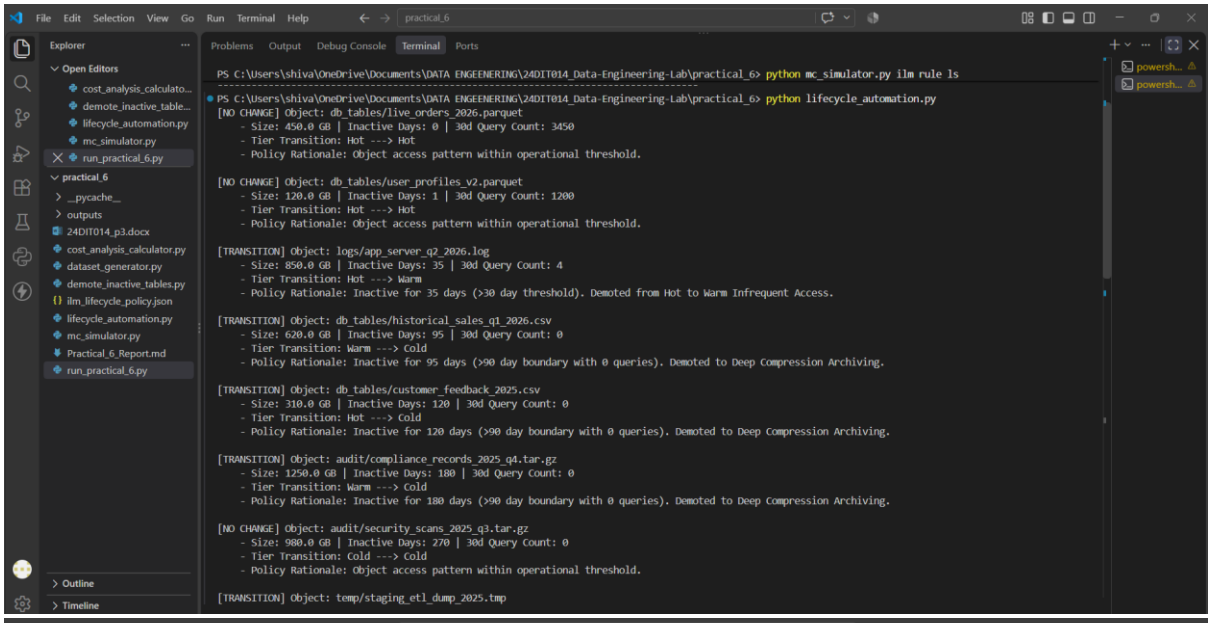
Supplementary Problem

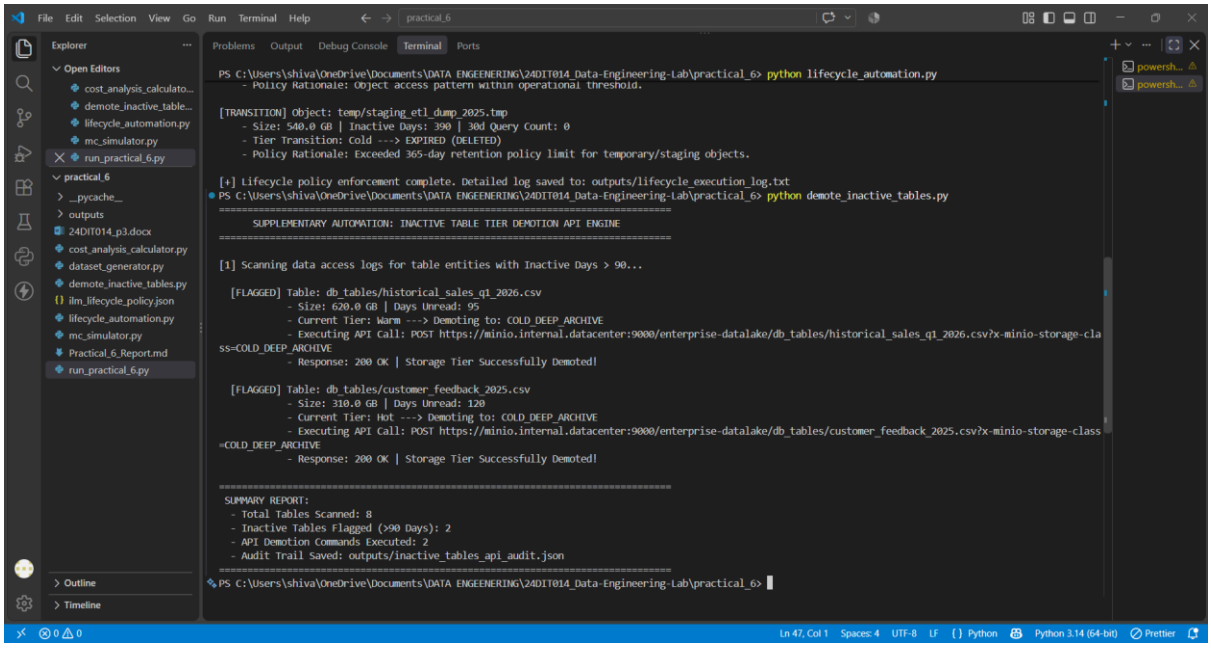
Create an automation script that evaluates data access logs, identifies data tables or objects that have not been read for more than 90 days, and automatically demotes their storage allocation to an appropriate Cold storage tier via API calls.

Output

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\shiva\OneDrive\Documents\DATA ENGEENERING\24DIT014_Data-Engineering-Lab\practical_6> python mc_simulator.py ilm rule ls
mc: Lifecycle rules configured on alias 'myminio/enterprise-datalake':
--------------------------------------------------------------------------------
RULE ID                          STATUS     PREFIX          TRANSITION / EXPIRATION
--------------------------------------------------------------------------------
TransitionToWarmInfrequentAccess Enabled    logs/           30d -> WARM_INFREQUENT
DemoteInactiveDataTablesToCold   Enabled    db_tables/      30d -> WARM_INFREQUENT, 90d -> COLD_DEEP_ARCHIVE
ArchiveComplianceAuditRecords    Enabled    audit/          60d -> WARM_INFREQUENT, 180d -> COLD_DEEP_ARCHIVE
ExpireTemporaryStagingDumps      Enabled    temp/           Expire after 365 days
--------------------------------------------------------------------------------
```





Final Conclusion

The practical demonstrates how automated storage lifecycle management can be used to organize data according to its access frequency and retention requirements. By dividing data into Hot, Warm, and Cold tiers, frequently accessed data can remain readily available while older and less frequently accessed data can be moved to cost-efficient storage.

The use of MinIO, lifecycle policies, JSON schemas, and Python automation provides a practical approach for implementing automated and cost-conscious object storage management.

GITHUB REPOSITORY

https://github.com/Jaykansar/D25DIT078_DATA-ENGINEERING_PRACTICALS/tree/main/Practical%206